BioHackathon series:
COST Action GOBLIN Hackathon
Oviedo, Spain
GOBLIN

Submitted: 27 Jul 2026

Published by BioHackrXiv.org

Schema-Driven Generation of Synthetic HL7 FHIR RDF Data from Shape Expressions (ShEx)

Diego Martín-Fernández [1], Álvaro García-Fernández [1], Samuel Bustamante-Larriet [1], and Eric Prud'hommeaux [2]

1 University of Oviedo **2** Independent

Abstract

We describe how synthetic HL7 FHIR data in RDF was produced directly from Shape Expressions (ShEx), using the authoritative FHIR R4 ShEx schema as the sole source of domain structure. Rather than encoding clinical knowledge in a domain-specific simulator, we drive generation from the published shapes: a schema-driven generator (*rudof generate*) consumes them, and a small configuration file controls scale, cardinalities, and value generation. This note reports the method—schema selection, a minimal schema preparation step, the generator configuration, and the invocation—so that the process is reproducible.

Introduction

HL7 FHIR (**fhir_r4:cite?**) is the *de facto* standard for exchanging electronic healthcare information, and it defines a canonical RDF representation of its resources (**solbrig_modeling_2017:cite?**). Synthetic FHIR data in RDF is useful for exercising triple stores, SPARQL endpoints, validators, and terminology services without touching real patient records. Producing such data usually relies on domain-specific simulators that encode clinical knowledge.

This note takes a different route: it generates synthetic FHIR R4 RDF directly from the FHIR ShEx schema using *rudof* (**labra_gayo_rudof_2024:cite?**), a schema-driven generator. Because the schema alone drives generation, the same tool that produces data for any ShEx or SHACL schema produces FHIR data, with no FHIR-specific implementation. The work was carried out as a use case at the GOBLIN hackathon (https://dmki-tuwien.github.io/goblin-hackathon).

Background: *rudof generate*

rudof (**labra_gayo_rudof_2024:cite?**) is a Rust toolkit for handling RDF data models and shapes (ShEx and SHACL (**validating_rdf_book:cite?**). Its generator, *rudof generate*, interprets a shapes schema as a specification of the instance data to produce: shape declarations become node types, and the triple constraints of each shape become the properties of its instances, with the declared cardinalities, datatypes, node kinds, and shape references governing how each instance is populated and linked.

Generation runs in two phases. In the first phase, one instance is created per requested entity and its *value* properties are filled: for each triple constraint whose object is a literal, the number of values is drawn from the declared cardinality according to the configured strategy, and each value is produced by the datatype's field generator. In the second phase, the *reference* properties—triple constraints whose object is another shape—are resolved by selecting, for

every reference, an existing instance of the target shape, so that no dangling references are emitted and referential integrity is preserved. When no random variation is configured, the structure of the output (entity counts, type assertions, and links) is a deterministic function of the schema and the configuration, and the total work is linear in the size of the generated graph. All of this behaviour is set by a single configuration file (Sect. Configuration).

Method

Input schema

We used the authoritative FHIR R4 ShEx schema (https://hl7.org/fhir/R4/fhir.shex) produced by the FHIR-to-ShEx build process [(**solbrig_modeling_2017:cite?**)](**fhir_r4:cite?**). It contains 684 shapes covering every R4 resource and datatype, so the generator operates under the same structural definitions the FHIR community uses for validation, without any extraction or approximation step.

Schema preparation

In the published schema the primitive leaves of FHIR datatypes do not carry an XSD type directly: each primitive (`string`, `date`, `code`, `integer`, ...) is expressed through the FHIR `fhirpath` system primitive shapes (`System.String`, `System.Date`, ...), whose value node is left abstract. A generator driven by these shapes has no XSD datatype to dispatch on, and would therefore emit an empty placeholder node at every leaf instead of a concrete literal. To obtain typed literal values we produced a minimally modified variant of the schema in which each `fhirpath` primitive shape is rebound to the corresponding XSD datatype (`System.String` → `xsd:string`, `System.Date` → `xsd:date`, `System.Integer` → `xsd:integer`, and so on). This rewrite touches only the primitive value nodes; the resource shapes, their properties, cardinalities, and references are left exactly as published, so the generated structure still follows the authoritative schema. With the leaves now typed, the per-datatype field generators of the configuration fire on every primitive and produce real values.

Configuration

A single TOML file configures the run. Its `[generation]` block sets the global generation policy and the `[field_generators]` blocks control how literal values are synthesized at the schema leaves. The parameters used were:

- `entity_count` and `seed` fix the number of top-level instances and the RNG seed, making runs reproducible.
- `entity_distribution = "Equal"` splits instances evenly across the shapes, so every FHIR resource type is represented instead of following a skewed distribution.
- `cardinality_strategy = "Maximum"` realizes each repeatable property up to its upper bound, and `property_fill_probability = 1.0` with `property_selection_strategy = "All"` populates every optional property of every shape; together they exercise the full breadth of the schema rather than a sparse subset.
- `ignore_min_cardinality = false` honours the lower bounds declared in the shapes, and `property_count_variance = 0.0` disables random variation, so the structure of the output is fully determined by the schema, the configuration, and the seed.
- `[field\_generators.default]` sets the default locale (`en`) and value fidelity (`quality = "High"`); each XSD datatype is then bound to a specific generator with its own parameters (numeric ranges and precision, date-year windows, a regular expression for `xsd:time`, and a URI generator for `xsd:anyURI`).

An abridged version of the file follows.

```toml
[generation]
entity_count                = 18000
seed                        = 42
entity_distribution         = "Equal"      # even split across shapes
cardinality_strategy        = "Maximum"    # fill repeatable props to the bound
property_fill_probability   = 1.0          # populate every optional property
property_selection_strategy = "All"
ignore_min_cardinality      = false
property_count_variance     = 0.0          # deterministic given the seed

[field_generators.default]
locale  = "en"
quality = "High"

[field_generators.datatypes."http://www.w3.org/2001/XMLSchema#string"]
generator = "string"

[field_generators.datatypes."http://www.w3.org/2001/XMLSchema#boolean"]
generator = "boolean"

[field_generators.datatypes."http://www.w3.org/2001/XMLSchema#integer"]
generator  = "integer"
parameters = { min = 0, max = 1000 }

[field_generators.datatypes."http://www.w3.org/2001/XMLSchema#decimal"]
generator  = "decimal"
parameters = { min = 0.0, max = 500.0, precision = 2 }

[field_generators.datatypes."http://www.w3.org/2001/XMLSchema#date"]
generator  = "date"
parameters = { start_year = 1940, end_year = 2026 }

[field_generators.datatypes."http://www.w3.org/2001/XMLSchema#dateTime"]
generator  = "datetime"
parameters = { start_year = 2010, end_year = 2026, include_time = true }

[field_generators.datatypes."http://www.w3.org/2001/XMLSchema#time"]
generator  = "pattern"
parameters = { regex = "(0[0-9]|1[0-9]|2[0-3]):[0-5][0-9]:[0-5][0-9]" }

[field_generators.datatypes."http://www.w3.org/2001/XMLSchema#anyURI"]
generator = "uri"
```

Two additional blocks, omitted above, complete the file: an `[output]` block that sets the output path, the serialization format (Turtle), and whether generation statistics are written; and a `[parallel]` block that controls worker threads and batch size and enables shape and field-level parallelism during generation. Neither affects the content of the dataset, only where it is written and how fast.

Execution

The dataset is produced with a single command (*rudof generate*, version 0.1.118). Because the entity count, seed, output path, and serialization format are already set in the configuration file, the invocation only needs the schema and the configuration:

```
rudof generate -s fhir_r4.shex -c fhir_config.toml
```

The same values can also be given on the command line (`-n`, `--seed`, `-o`, `--result-format`); when present, the command-line flags override the configuration. We keep them in the TOML instead, so that a run is fully described by the schema and a single file.

Outcome

The run yields a FHIR R4 RDF (Turtle) dataset that exercises all 145 R4 resource types, at a scale set by the entity count (here, 18,000 entities). Because the structure of the output is a deterministic function of the schema, the configuration, and the seed, the dataset can be regenerated exactly rather than stored, and its scale and density can be adjusted by editing the configuration alone.

Availability. The modified FHIR ShEx schema variant, the generation configuration, the exact invocation, and the generated FHIR R4 RDF are available in the benchmark repository (https://github.com/DiegoMfer/benchmarks-synthetic-data-generators).

Conclusion

Synthetic FHIR R4 RDF can be generated from the published FHIR ShEx schema alone, using *rudof generate* and a short configuration file, without writing any domain-specific code. The approach is reproducible and configurable, and reduces the generation of FHIR data to schema selection, a one-line primitive rebinding, and a single command. Ongoing work on a FHIR-aware mode (`rudof-fhir` https://github.com/rudof-project/rudof-fhir) aims to emit the canonical FHIR-RDF serialization idioms directly.

References

Author contributions

Diego Martín-Fernández: Conceptualization, Software, Writing – original draft Álvaro García-Fernández: Conceptualization, Writing – review & editing Samuel Bustamante-Larriet: Writing - review & editing Eric Prud'hommeaux: Conceptualization - review